LAB

Integrate dynamic knowledge sources to enhance RAG output quality

IBM SkillsBuild

Introduction

In this lab, you will use dynamic, that is, frequently updated external knowledge sources to enhance RAG output quality.

Software requirements

To complete this lab, you don’t have to install any software on your computer. You only require a Google account to access Google Colab and a Replicate account to access and use various artificial intelligence (AI) models.

Objective

After completing this lab, you should be able to:

- Integrate dynamic knowledge sources to enhance RAG output quality

Lab steps

This lab requires you to complete the following steps:

- Create a GitHub account
- Create a Replicate account
- Sign up for Google Colab
- Prepare the Google Colab notebook with the necessary libraries
- Extract content from dynamic knowledge sources through web crawling
- Test the system to assess the response quality

Estimated duration to complete

30 minutes

Scenario
Background information

The customer support agents at a telecom company use a RAG-based assistant to answer customer questions. While the assistant extracts relevant details from the company’s internal support manuals, it sometimes misses details about new or emerging issues. In such cases, the agents refer to online community forums for the latest troubleshooting tips.

Challenge

The information on these online community forums often falls into one of the following categories:

- Unverified
- Incomplete
- Contradictory

This can lead to inconsistent answers and reduced customer trust.

Solution

To address these challenges, you propose enhancements to the RAG system used by the customer support team by integrating it with a dynamic data source. This will allow the team to access continuously updated information instead of details extracted from static internal manuals. In this lab, you’ll implement and test these enhancements.

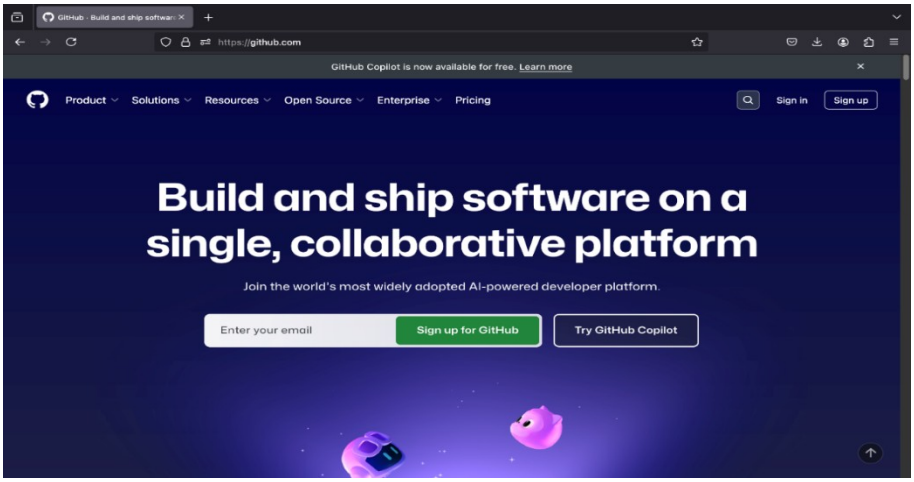
Create a GitHub account

Overview

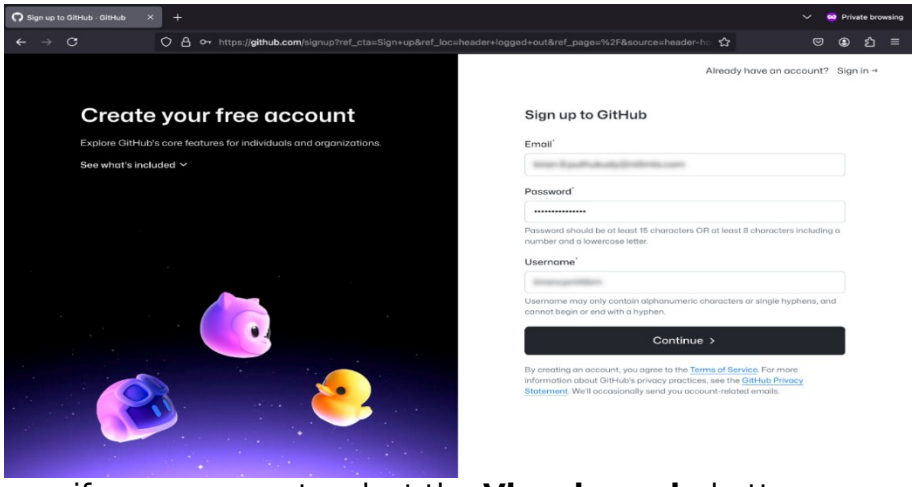
In this procedure, you'll set up a GitHub account. GitHub is a platform that helps developers store, manage, and share code, while also supporting collaboration through tools such as version control, bug tracking, and task management. Setting up a GitHub account provides access to the Replicate platform, which is required to complete the lab efficiently.

Instructions

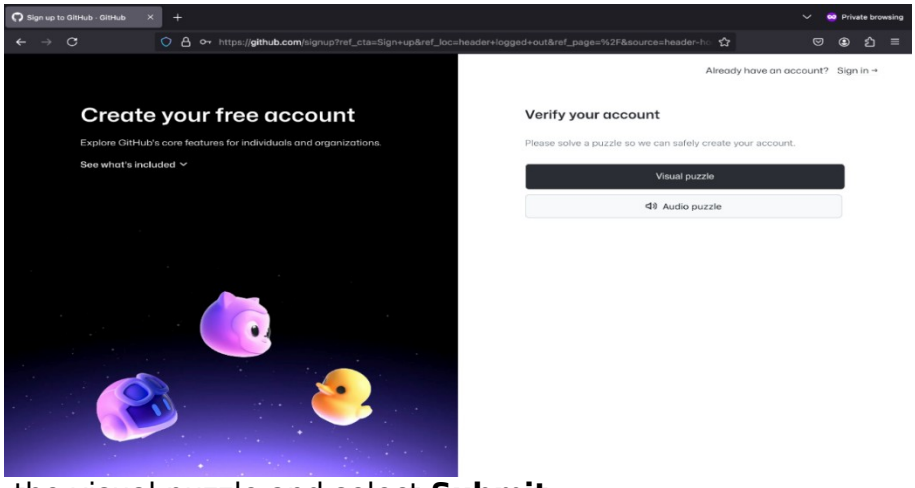
1. To create a GitHub account, go to the GitHub website.
2. Select the **Sign up for GitHub** button, as shown in the following screenshot.



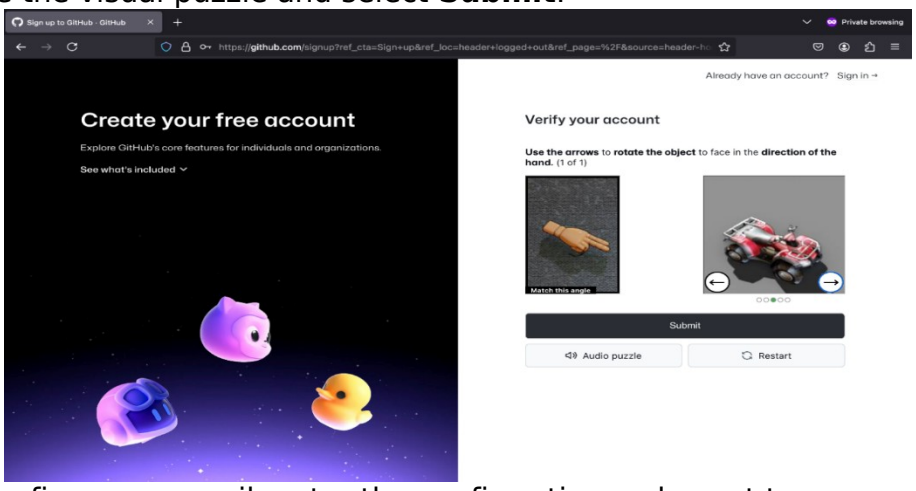
3. Enter your details in the **Email**, **Password**, and **Username** fields. Then, select **Continue**.
   The following screenshot shows the "Sign up to GitHub" page.



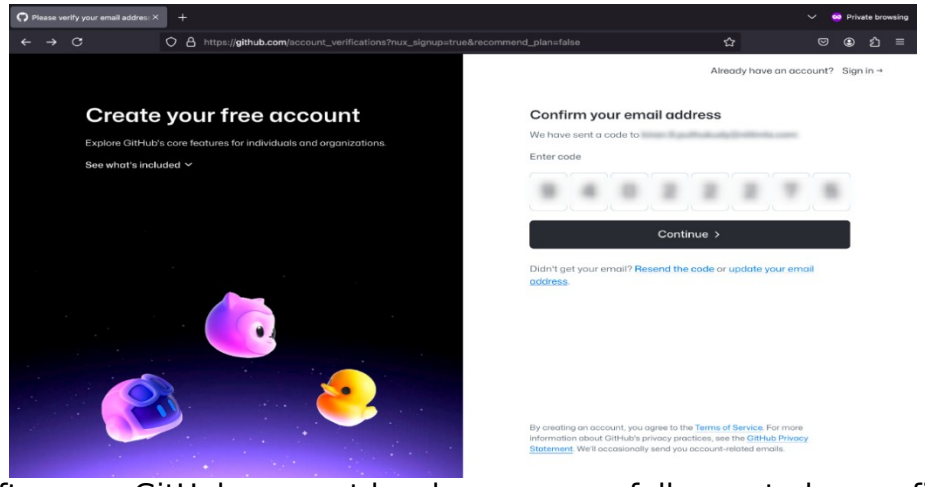
4. To verify your account, select the **Visual puzzle** button.



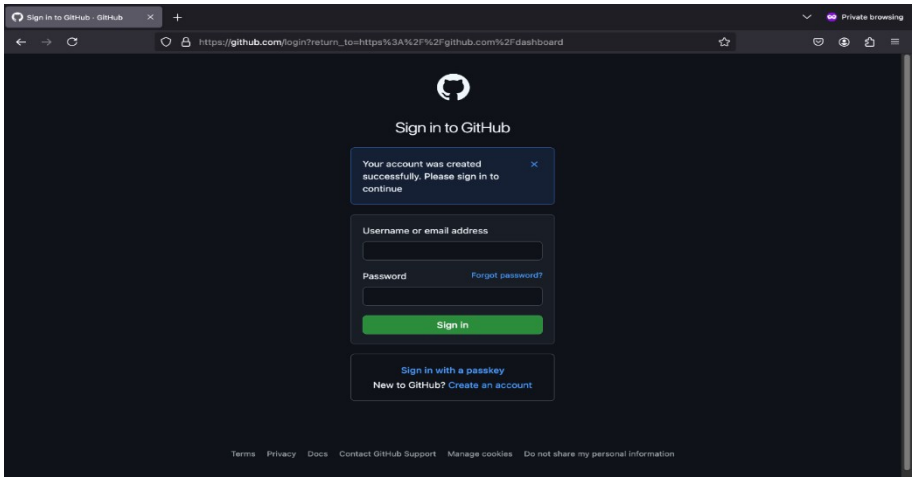
5. Solve the visual puzzle and select **Submit**.



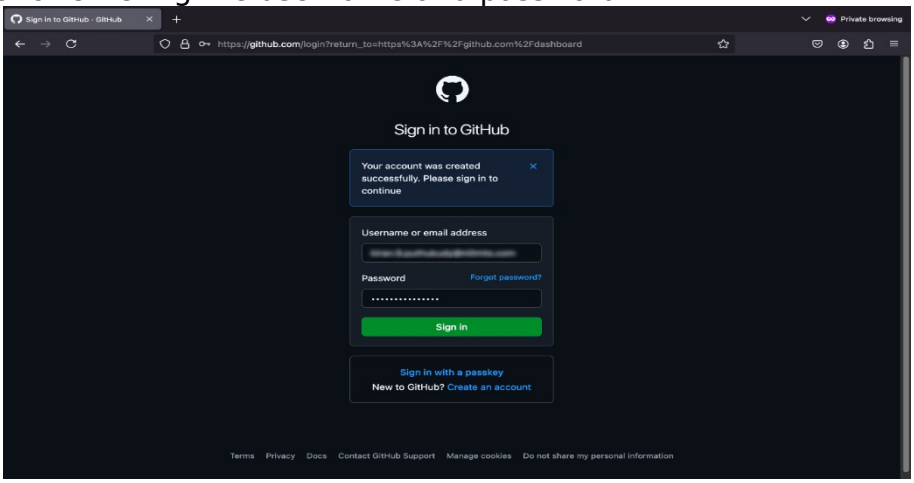
6. To confirm your email, enter the confirmation code sent to your registered email in the **Enter code** field and select **Continue**.



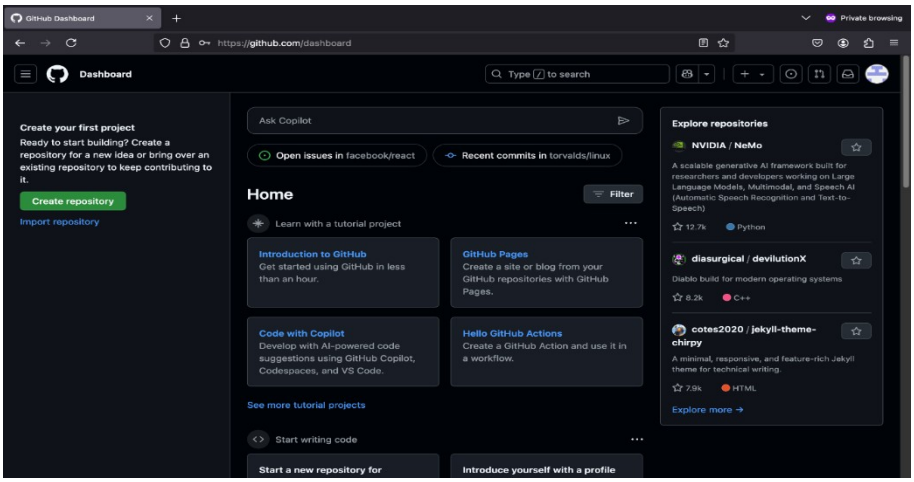
7. After your GitHub account has been successfully created, a confirmation message is displayed as shown in the following screenshot.



8. To sign in to your account, enter your credentials in the **Username or email address** and **Password** fields, and then select **Sign in**.
The following screenshot shows the GitHub sign-in page which includes fields for entering the username and password.



9. After you sign in, the GitHub dashboard appears as shown in the following screenshot.



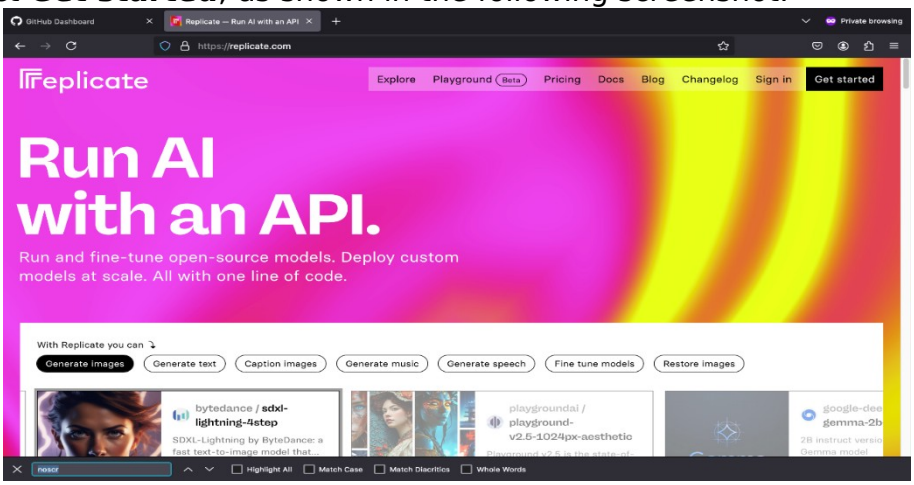
Create a Replicate account

Overview

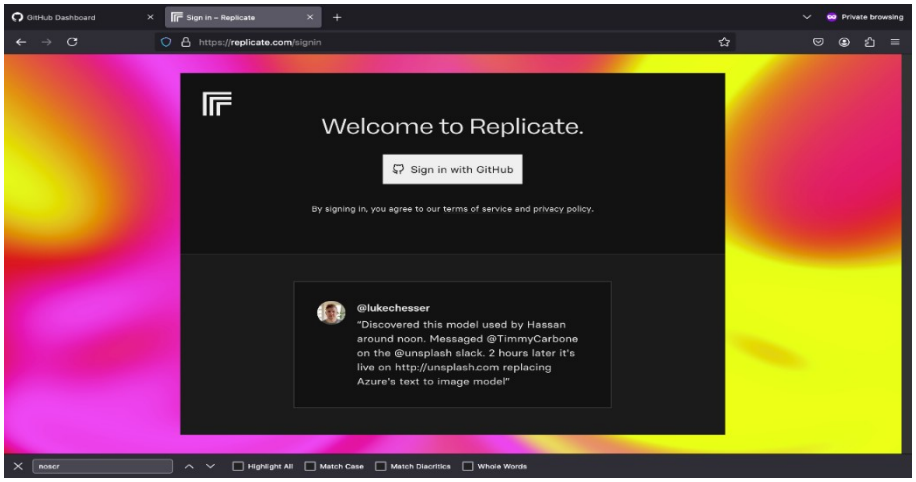
In this procedure, you'll use your GitHub account to register for a Replicate account. Replicate is a cloud-based platform that lets you use AI models such as IBM Granite without requiring advanced hardware. As part of this procedure, you'll create a Replicate token. A token is a secure access key that allows the lab environment to authenticate with Replicate and run models from your account in Google Colab.

Instructions

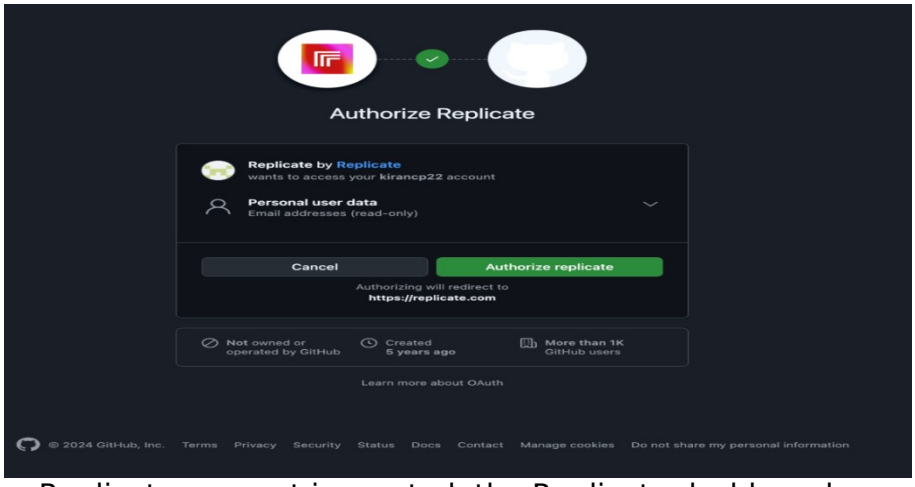
1. Go to the Replicate website.
2. Select **Get started**, as shown in the following screenshot.



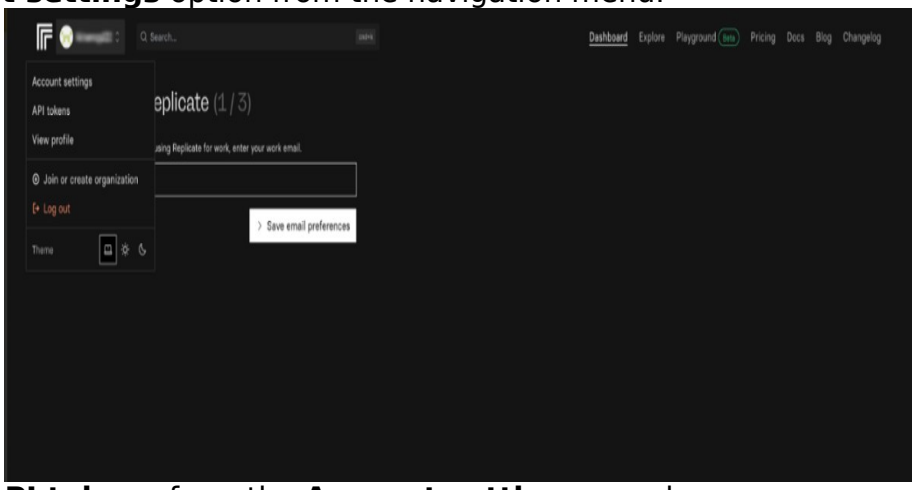
3. Select the **Sign in with GitHub** button, as shown in the following screenshot.



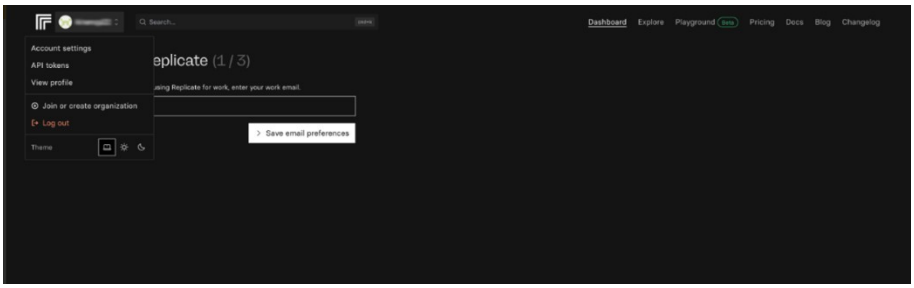
4. Select **Authorize replicate** to continue, as shown in the following screenshot.



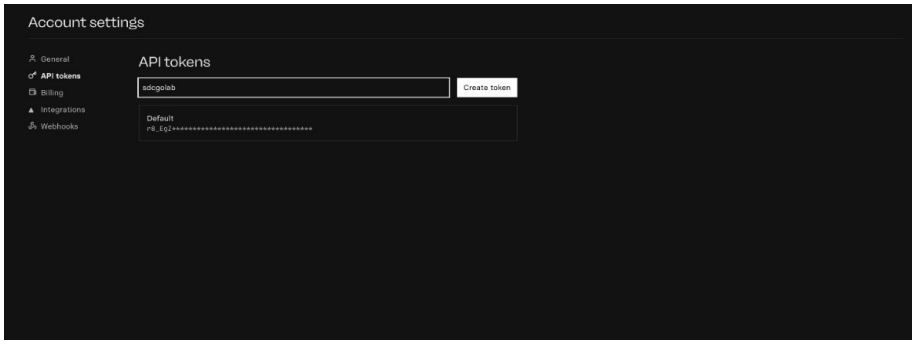
5. After your Replicate account is created, the Replicate dashboard appears as shown in the following screenshot. To create a Replicate token, select the **Account settings** option from the navigation menu.



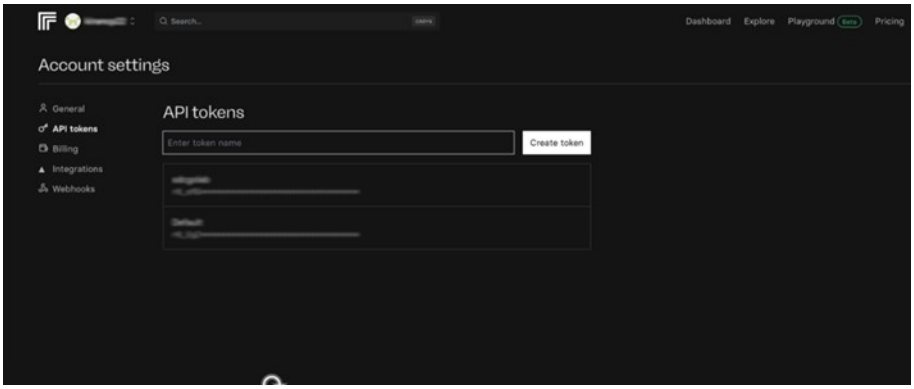
6. Select **API tokens** from the **Account settings** panel.
The following screenshot shows the Replicate dashboard.



7. Enter a name for the token in the **API tokens** field and then select **Create token**, as shown in the following screenshot.

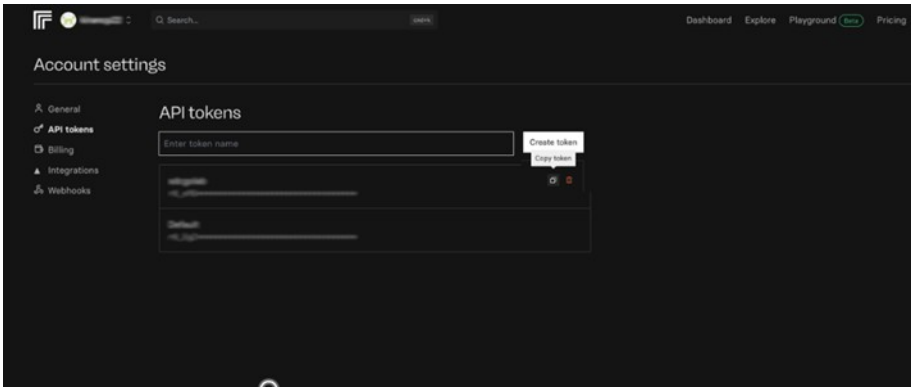


8. After your API token is created, it should be displayed on the “Account settings” page as shown in the following screenshot.



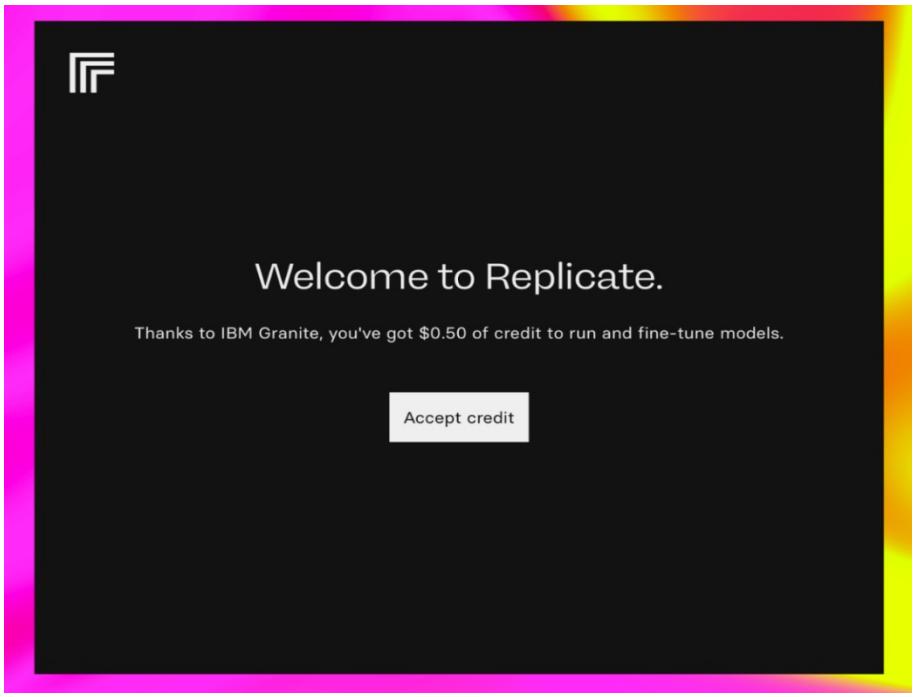
9. To copy the Replicate API, select the **Copy token** icon, as shown in the following screenshot.

Note: Save the Replicate token because you’ll need it to authenticate with the Replicate API when running code in the Google Colab environment later in this lab.

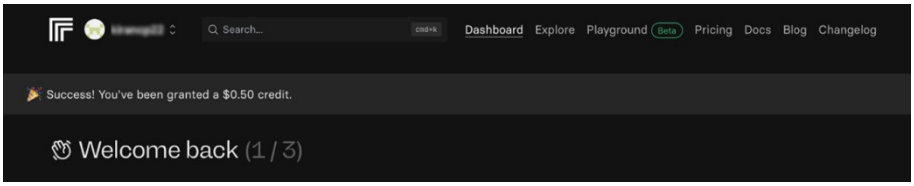


10. To run the lab without interruptions, you'll require some Replicate credits. Go to the Replicate invite link to claim your free $0.50 credit.

11. To claim the amount, select **Accept credit**.



12. A confirmation message is displayed indicating that a $0.50 credit has been added to your Replicate account as shown in the following screenshot.



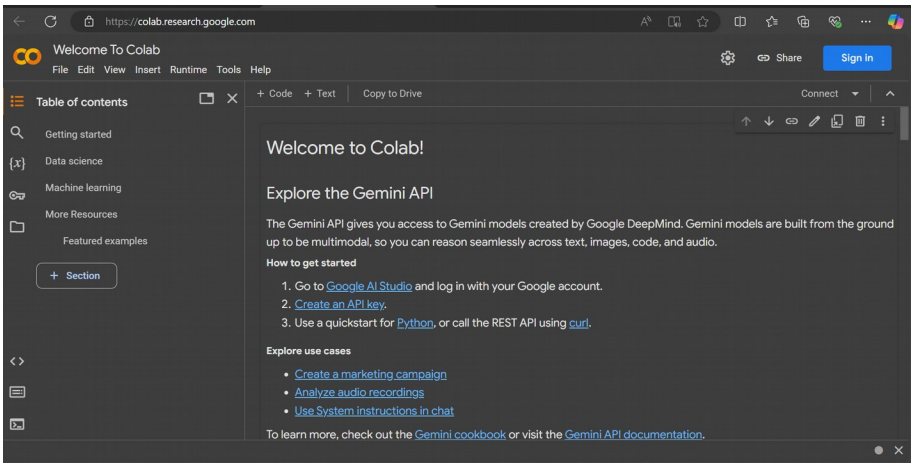
Sign up for Google Colab

Overview

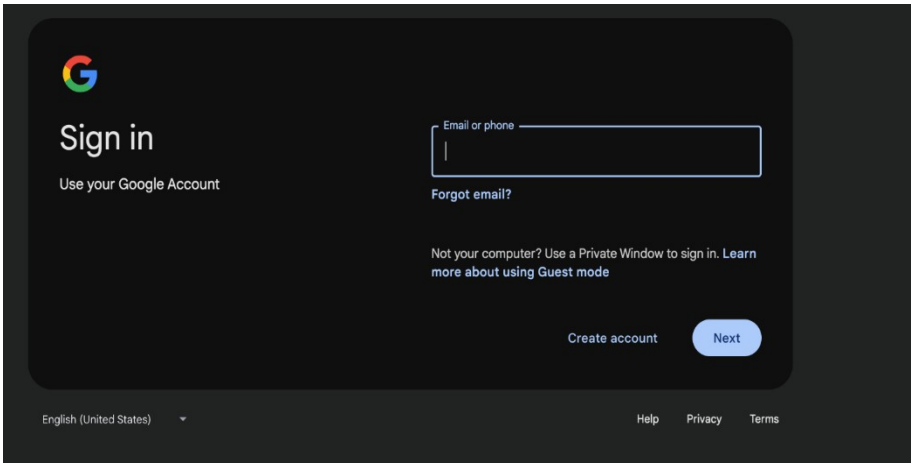
In this procedure, you'll set up a Google Colab account. Google Colab is a free cloud platform that lets you run code in notebooks, which are commonly used for machine learning, data science, and AI tasks. A Google Colab account allows you to install and use the tools needed to complete this lab.

Instructions

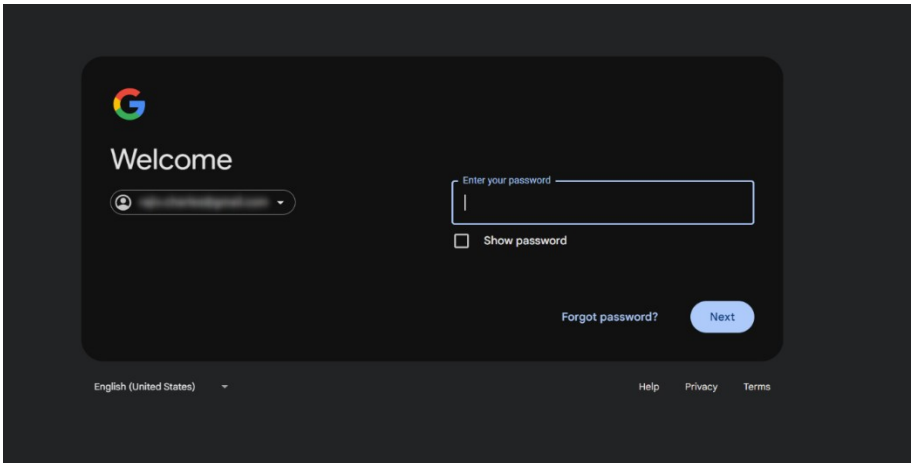
1. To sign up, go to the Google Colab website.
2. Select **Sign in**, as shown in the following screenshot.



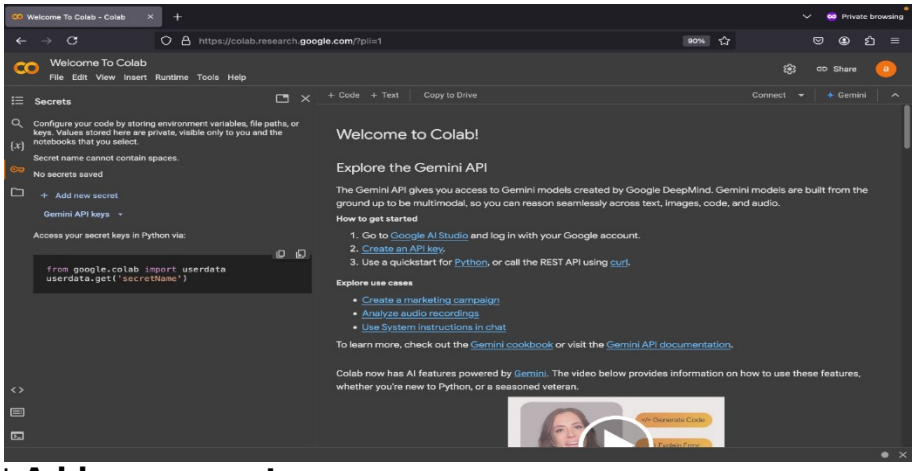
3. Enter your email or phone number in the **Email or phone** field, and then select **Next**.
   The following screenshot shows the Google sign-in page.



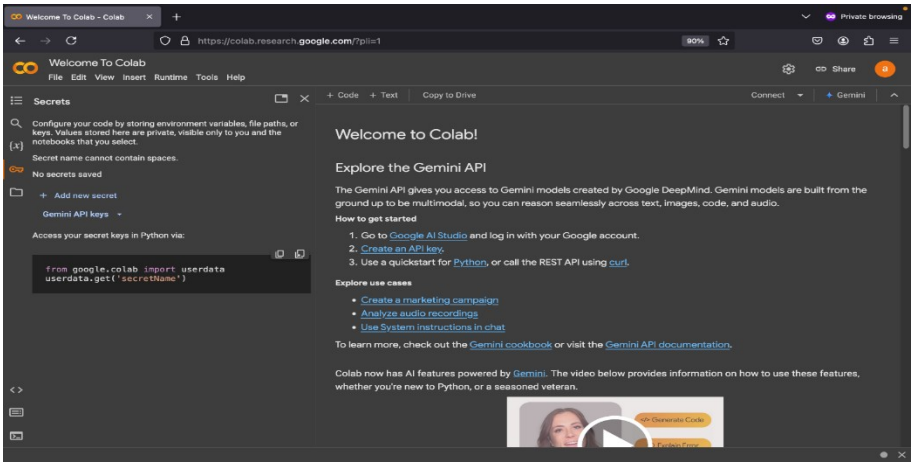
4. Enter your password in the **Enter your password** field, and then select **Next**.



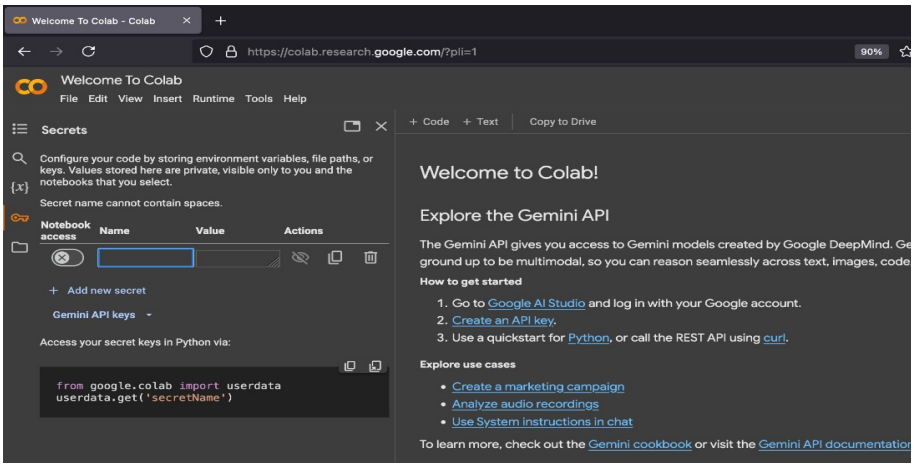
5. To store your Replicate API token, select the key icon from the **Secrets** tab on the Welcome to Colab page, as shown in the following screenshot.



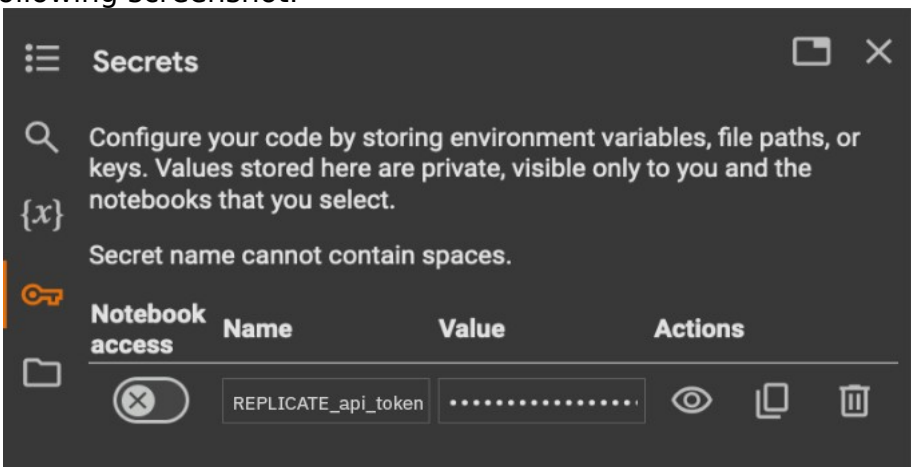
6. Select **Add new secret**.



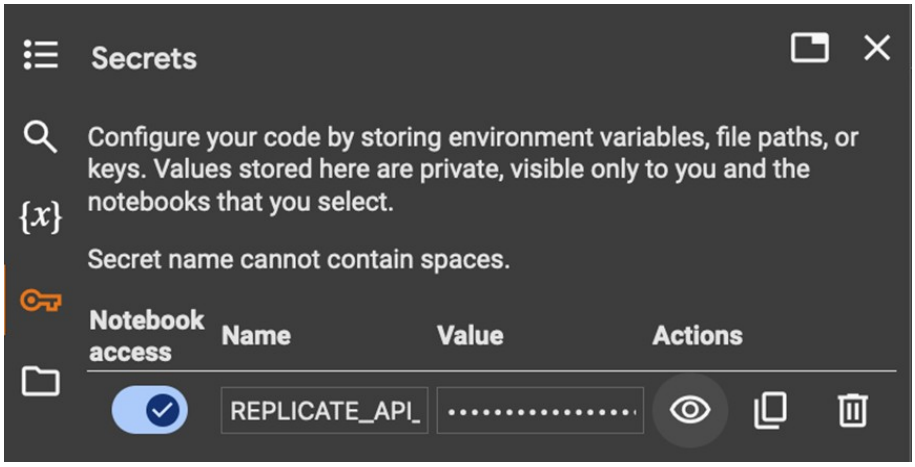
7. Type REPLICATE_api_token in the **Name** field.



8. Paste your Replicate API token you copied into the **Value** field, as shown in the following screenshot.



9. Set the **Notebook access** switch on, as shown in the following screenshot. Then, select the close icon to exit the configuration.



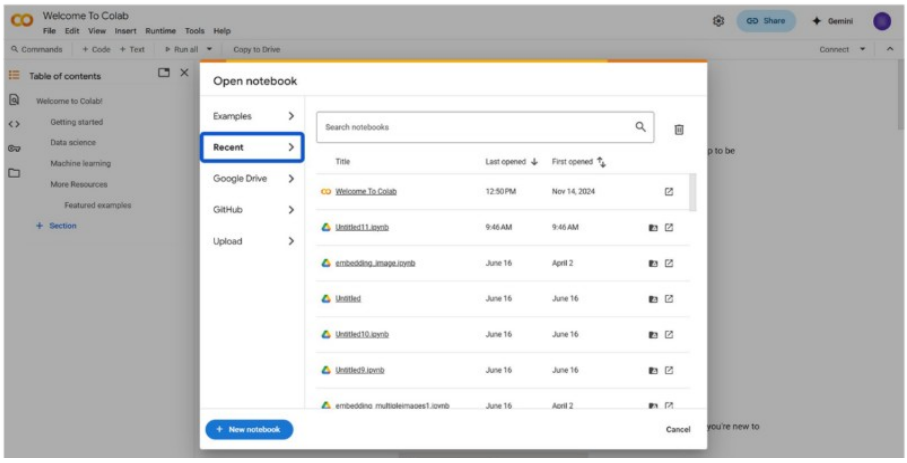
Prepare the Google Colab notebook with the necessary libraries

Overview

In this procedure, you'll create a new Google Colab notebook, install the required libraries, and import the necessary modules to prepare a workspace for the RAG system.

Instructions

1. Go to the Google Colab website. The Open notebook window is displayed.



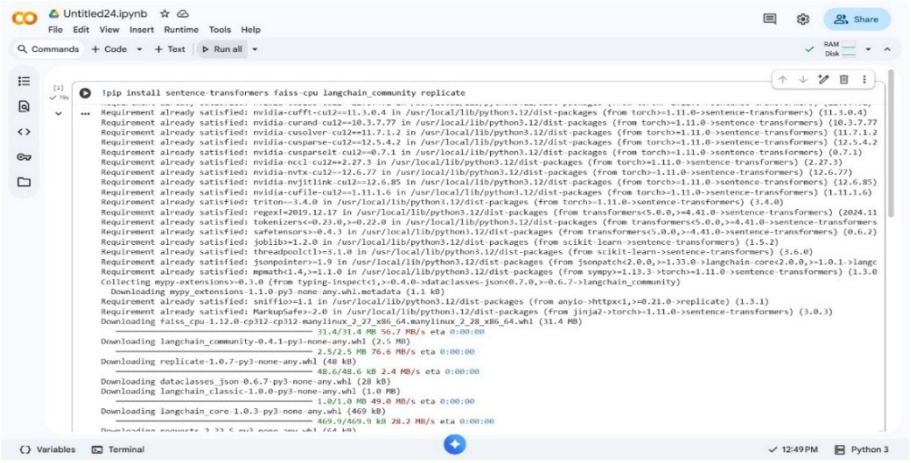
2. Select **New notebook**.

   Result: A new notebook with the first code cell is displayed, as shown in the following screenshot.

3. To retrieve accurate information and generate consistent, context-aware responses, you need to install four Python libraries: sentence-transformers, faiss-cpu, langchain_community, and replicate. These libraries might not be available in Google Colab by default. To install them, copy the following code, paste it into a new code cell, and then, select the **Play** icon to execute it.

```
#pip install sentence-transformers faiss cpu langchain_community replicate
```

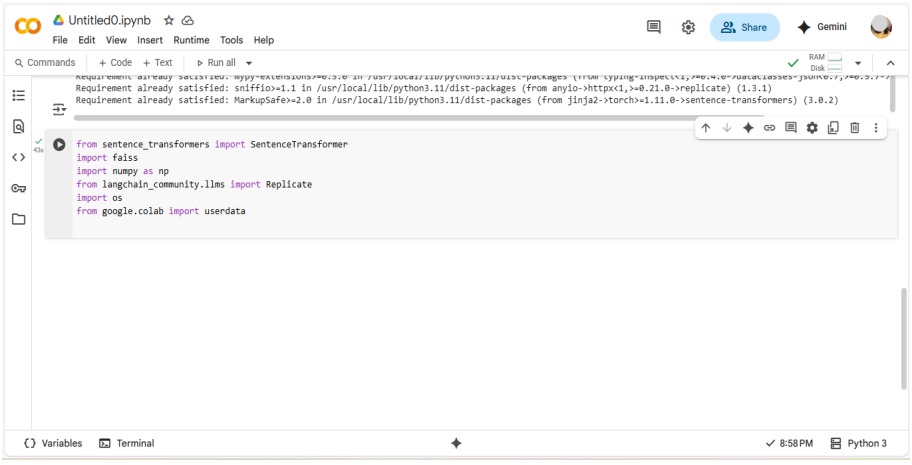
Result: The model generates the output that confirms you successfully installed required libraries as shown in the following screenshot.



4. Import the installed libraries to load them into your notebook. To do this, copy the following code, **paste** it into the code cell, and then select the **Play** icon to execute it.

```
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np
from langchain_community.llms import Replicate
import os
from google.colab import userdata
```

Result: Note that this code only loads the libraries into the notebook's memory and sets up the environment so there isn't a visible output. A green checkmark next to the Play icon indicates that the code ran successfully.



Extract content from a dynamic knowledge source through web crawling

Overview

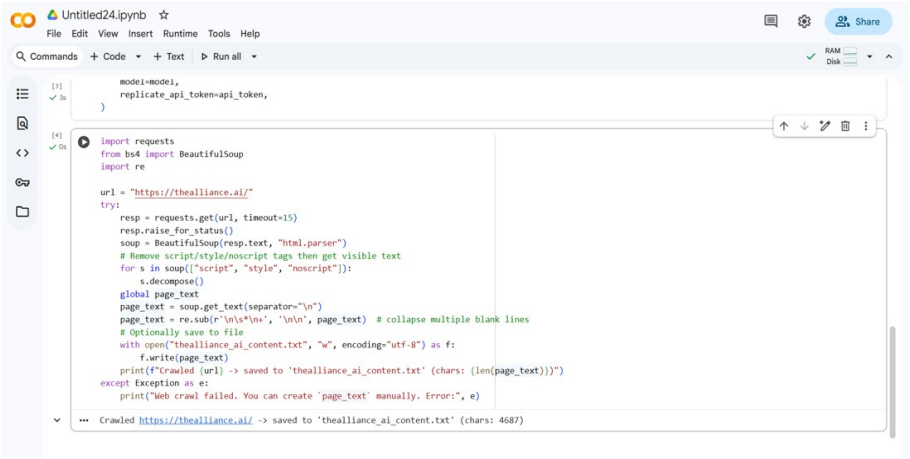
In this procedure, you'll integrate a dynamic knowledge base into your RAG system. To do so, you'll first gather content using the web crawling technique. Then, you'll chunk it, create embeddings for the chunked content, and store the chunks in a Facebook AI Similarity Search (FAISS) database.

Instructions

1. Begin gathering the content from a website using web crawling technique. To add the code for doing so, copy the following code, paste it into a new code cell, and then, select the **Play** icon to execute it.

```
import requests
from bs4 import BeautifulSoup
import re
url = "https://thealliance.ai/"
try:
    resp = requests.get(url, timeout=15)
    resp.raise_for_status()
    soup = BeautifulSoup(resp.text, "html.parser")
    # Remove script/style/noscript tags then get visible text
    for s in soup(["script", "style", "noscript"]):
        s.decompose()
    global page_text
    page_text = soup.get_text(separator="\n")
    page_text = re.sub(r'\n\s*\n+', '\n\n', page_text)  # collapse multiple blank lines
    # Optionally save to file
    with open("thealliance_ai_content.txt", "w", encoding="utf-8") as f:
        f.write(page_text)
    print(f"Crawled {url} -> saved to 'thealliance_ai_content.txt'
(chars: {len(page_text)})")
except Exception as e:
    print("Web crawl failed. You can create `page_text` manually. Error:", e)
```

Result: A green check mark confirms the code ran successfully. Note that the output shows the web content from thealliance.ai website was saved to thealliance_ai_content.txt.

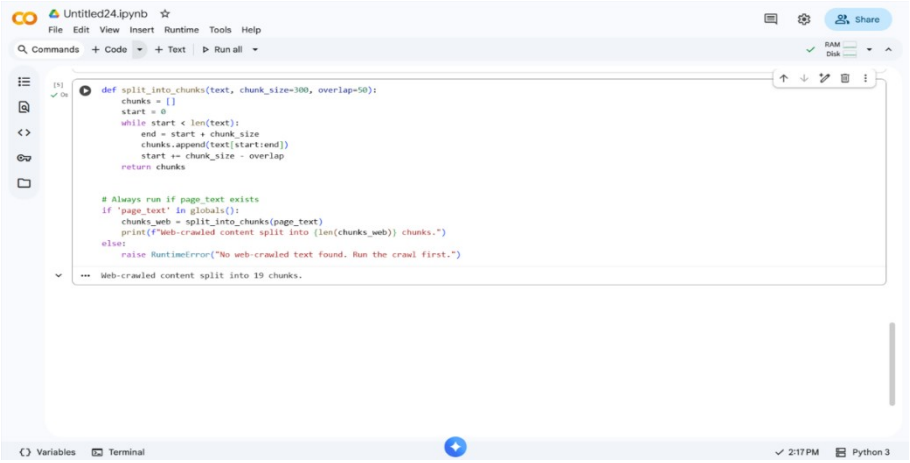


Note: Web crawling on random websites might interfere with their usage and privacy Terms and Conditions. For this lab, consent has been obtained from theallaince.ai.

2. Chunk the content in thealliance_ai_content.txt. To do so, copy the following code, paste it into a new code cell, and then, select the **Play** icon to execute it.

```
# Create chunks for web crawl
def split_into_chunks(text, chunk_size=300, overlap=50):
   chunks = []
   start = 0
   while start < len(text):
      end = start + chunk_size
      chunks.append(text[start:end])
      start += chunk_size - overlap
   return chunks
# Always run if page_text exists
if 'page_text' in globals():
   chunks_web = split_into_chunks(page_text)
   print(f"Web-crawled content split into {len(chunks_web)} chunks.")
    else:
   raise RuntimeError("No web-crawled text found. Run the crawl first.")
```

Result: A green check mark confirms the code ran successfully. Notice the message that the document is split into 19 chunks.

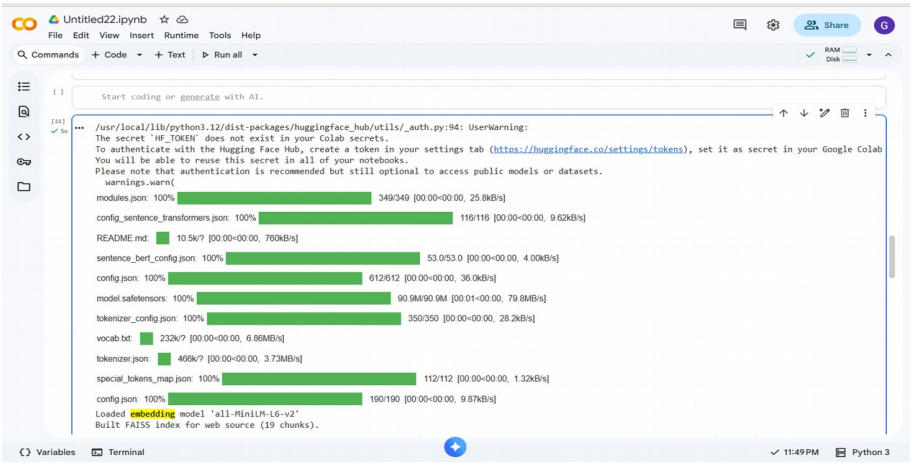


3. Convert the content chunks into vector embedding. To do so, copy the following code, paste it into a new code cell, and then, select the **Play** icon to execute it.

```
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np
# Ensure embedding model (will reuse if already present)
try:
    embed_model
except NameError:
    embed_model = SentenceTransformer('all-MiniLM-L6-v2')
    print("Loaded embedding model 'all-MiniLM-L6-v2'")
def build_index_from_chunks(chunks_local):

  embs = embed_model.encode(chunks_local, convert_to_numpy=True).astyp
e(np.float32)
    dim = embs.shape[1]
    idx = faiss.IndexFlatL2(dim)
    idx.add(embs)
    return idx, embs
# Build web index if needed
if 'chunks_web' in globals():
    if 'index_web' not in globals():
        index_web, embeddings_web = build_index_from_chunks(chunks_web)
        chunk_store_web = chunks_web
        print(f"Built FAISS index for web source ({len(chunks_web)} chunks).")
    else:
        print("Web FAISS index already present.")
```

Result: This code converts each chunk into an embedding using a sentence-transformer model. It also stores the embeddings in a searchable FAISS database. A green check mark confirms that the code ran successfully. Note the output showing that all embeddings inserted into the FAISS index, with the total count.



You've integrated the RAG system with live data from thealliance.ai and so you decide to test the RAG system.

Test the system to assess the response quality

Overview

In this procedure, you'll test your RAG system. For this, you'll enter a query, just like an end user would, and assess the output the RAG system generates.

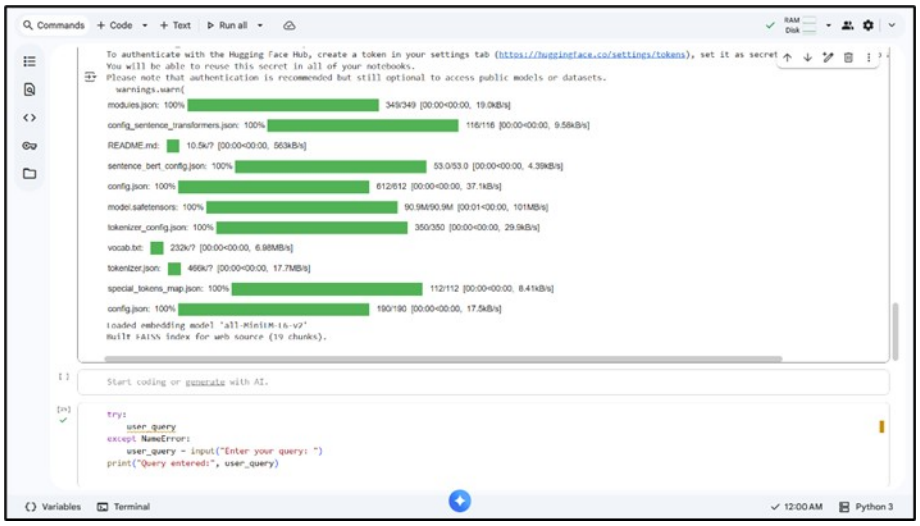
Instructions

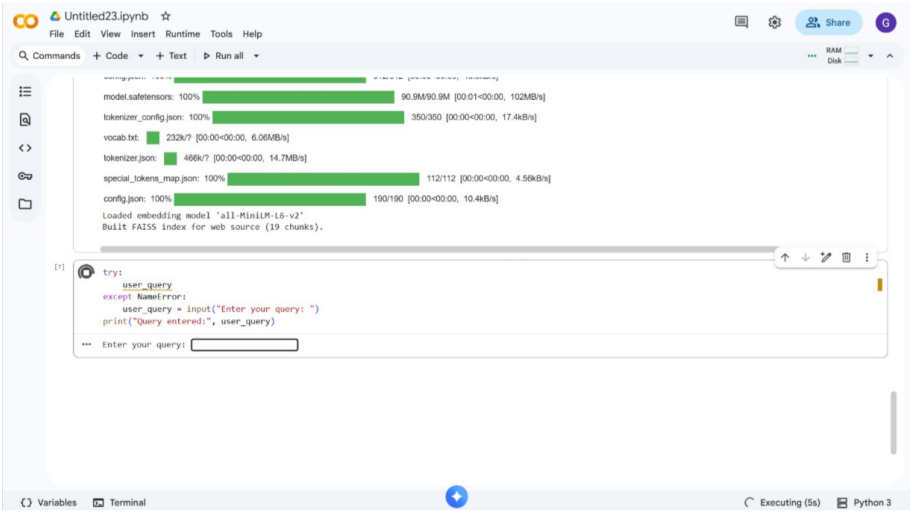
1. Add code that prompts users to input a query. To do so, copy the following code, paste it into a new code cell, and then, select the **Play** icon to execute the code.

```
try: user_query
except NameError:
  user_query = input("Enter your query: ")
print("The query you entered: ", user_query)
```

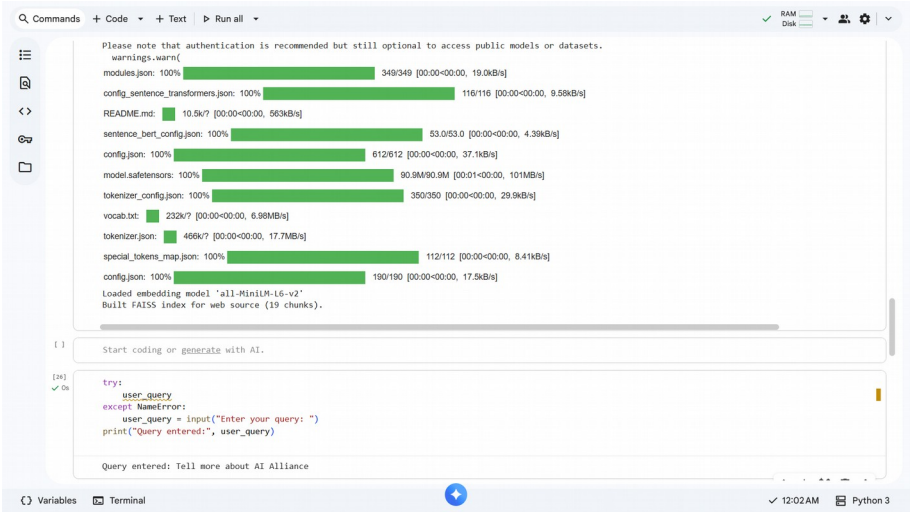
Result: The code executes and you are prompted to ask a question.

2. In the **Enter your query** field, type, Tell more about AI Alliance. Then, press **Enter**.





Result: The model displays the query you entered.



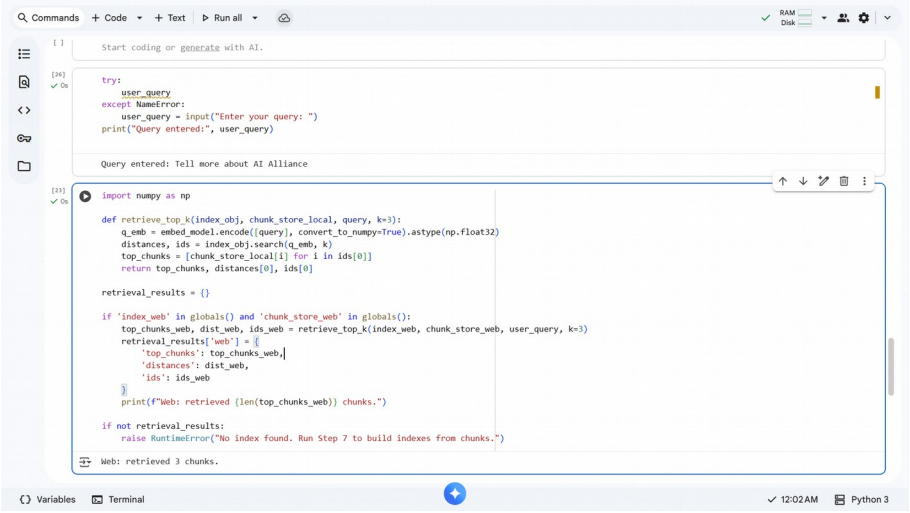
3. Retrieve the top three responses to test your system. Retrieving only the top few results is a best practice that helps keep the context focused, reduces noise, and allows the model to generate more accurate and concise answers. To do so, copy the following code, paste it into a new code cell, and then, select the **Play** icon to execute the code.

```
import numpy as np
def retrieve_top_k(index_obj, chunk_store_local, query, k=3):

 q_emb = embed_model.encode([query], convert_to_numpy=True).astype(np
.float32)
   distances, ids = index_obj.search(q_emb, k)
   top_chunks = [chunk_store_local[i] for i in ids[0]]
   return top_chunks, distances[0], ids[0]
retrieval_results = {}
if 'index_web' in globals() and 'chunk_store_web' in globals():

 top_chunks_web, dist_web, ids_web = retrieve_top_k(index_web, chunk_stor
e_web, user_query, k=3)
   retrieval_results['web'] = {
      'top_chunks': top_chunks_web,
      'distances': dist_web,
      'ids': ids_web
   }
   print(f"Web: retrieved {len(top_chunks_web)} chunks.")
if not retrieval_results:
    raise RuntimeError("No index found. Run Step 7 to build indexes from
chunks.")
```

Result: A green check mark confirms the code ran successfully. Note that the output message lists the number of retrieved chunks.



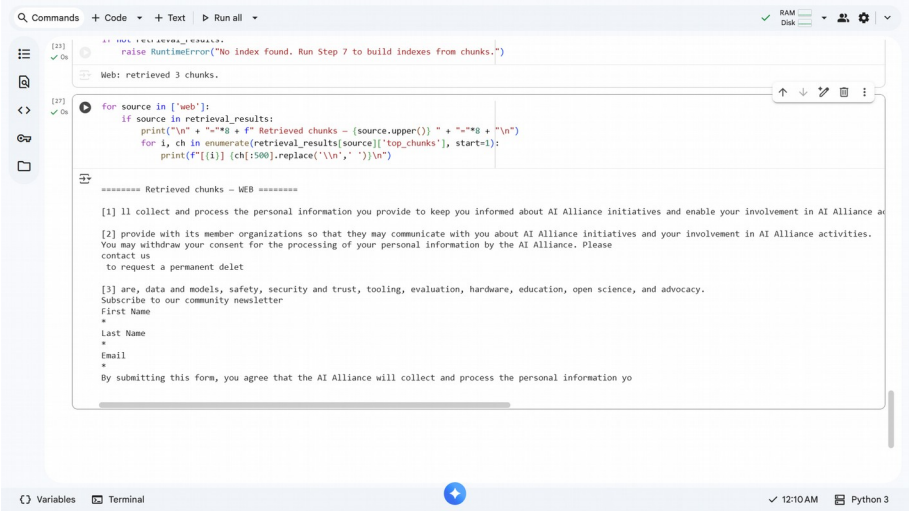
4. Review the retrieved chunks to verify whether the RAG system is extracting the latest information in response to your query. To do so, copy the following

code, paste it into a new code cell, and then, select the **Play** icon to execute the code.

```
for source in ['web']:

 if source in retrieval_results:
 print("\n" + "="*8 + f" Retrieved chunks — {source.upper()} " + "="*8 + "\
n")
 for i, ch in enumerate(retrieval_results[source]['top_chunks'], start=1):
 print(f"[{i}] {ch[:500].replace('\\n',' ')}\n")
```

Result: A green check mark confirms that the code ran successfully. The output displays the top 3 chunks retrieved by the RAG system.



Result: Your RAG system is working as intended. You entered the query “Tell more about AI Alliance” into the system. The top three response to this query is relevant information about the AI Alliance website. These responses were extracted from the dynamic content that was gathered from thealliance.ai. These test query verifies that integrating dynamic content from sources external to your RAG system will make the assistant’s responses more current and comprehensive.

Conclusion

Congratulations! You set up the Google Colab notebook, extracted content from thealliance.ai using web crawling, and tested your RAG system with a query. You've confirmed that integrating dynamic knowledge sources works, and now it’s time to enhance your RAG system with customer support data. By adding these sources, your team will access accurate, continuously updated information instead of static manuals, enabling faster, more precise, and context-aware answers that boost customer satisfaction.